10 Python Case Studies

Syllabus-based solutions with algorithms, flowcharts, pseudocode, code, test cases and output screenshots

Topics used: computational thinking, algorithms, flowcharts, pseudocode, variables, data types, operators, conditional statements, loops, strings, pattern generation, lists, nested lists, tuples, tuple operations and unpacking.

Flowchart color legend

Symbol	Meaning	Color
Rounded terminator	Start / Stop	Green / light red
Parallelogram	Input / Output	Light blue
Rectangle	Process / calculation	Light yellow
Diamond	Decision	Light orange
Arrow	Direction of flow	Dark grey

Important note: The compact programs are educational demonstrations. Real-world systems need stronger input validation, persistent storage, security and institution-specific rules. Sample rates and thresholds are assumptions for practice, not official policies. The output images are terminal-style sample previews created for this report, not screenshots of a live Python interpreter.

1. University Scholarship Allocation

Problem statement: Evaluate scholarship applicants using aggregate marks, subject-wise minimum marks, attendance, and family income. Rank eligible students and apply tie-break rules.

Objectives

• Validate applicant criteria
• Calculate eligibility and rank students
• Resolve ties by aggregate marks, attendance, then lower income

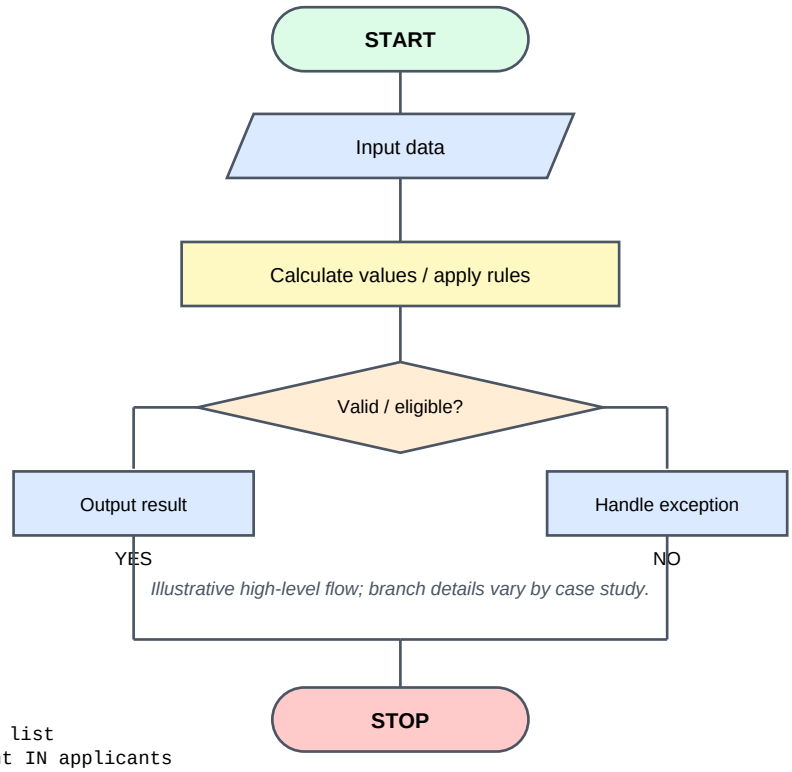
Assumptions

• Marks and attendance are percentages from 0 to 100.
• Eligibility: aggregate ≥ 80, every subject ≥ 60, attendance ≥ 75, annual income ≤ ■300000.
• Applicants are represented as tuples: (name, aggregate, subject_marks, attendance, income).

Algorithm

1. Read the list of applicants.
2. For each applicant, check every eligibility condition.
3. Store eligible applicants with their ranking values.
4. Sort by aggregate descending, attendance descending, income ascending.
5. Display ranked applicants or a no-eligible-applicants message.

Flowchart



Illustrative high-level flow; branch details vary by case study.

Pseudocode

```
START
INPUT applicants
eligible ← empty list
FOR each applicant IN applicants
  IF aggregate ≥ 80 AND minimum(subject_marks) ≥ 60 AND attendance ≥ 75 AND income ≤ 300000
    ADD applicant TO eligible
SORT eligible BY aggregate DESC, attendance DESC, income ASC
OUTPUT eligible
STOP
```

Python source code (compact)

```
applicants = [("Asha", 88, [80, 75, 90], 92, 250000), ("Ravi", 82, [65, 70, 60], 80, 200000), ("Mina", 79, [90, 90, 90], 95, 15
eligible = [a for a in applicants if a[1] >= 80 and min(a[2]) >= 60 and a[3] >= 75 and a[4] <= 300000]
eligible.sort(key=lambda a: (-a[1], -a[3], a[4]))
print([(a[0], a[1]) for a in eligible] or "No eligible applicants")
```

Test cases

Input applicants	Expected output
Asha 88, subjects 80/75/90, attendance 92, income 250000	Eligible, rank 1

Input applicants	Expected output
Ravi 82, subjects 65/70/60, attendance 80, income 200000	Eligible, rank 2
Mina aggregate 79	Not eligible

Sample output

```
[('Asha', 88), ('Ravi', 82)]
```

SAMPLE OUTPUT PREVIEW | NOT A LIVE TERMINAL CAPTURE

```
Case Study 1: University Scholarship Allocation

> python scholarship.py

Eligible applicants (ranked):

1. Asha - Aggregate: 88

2. Ravi - Aggregate: 82

Applicants failing any threshold are excluded.
```

Conclusion

The system checks all scholarship rules, excludes applicants who fail any threshold, and ranks eligible applicants using deterministic tie-break criteria.

2. Progressive Electricity Billing

Problem statement: Calculate monthly household electricity bills using progressive consumption slabs, fixed charges, surcharge and rebate, then flag unusual month-to-month increases.

Objectives

• Calculate slab-wise energy charges
• Apply surcharge and rebate
• Compare bills and identify increases above a threshold

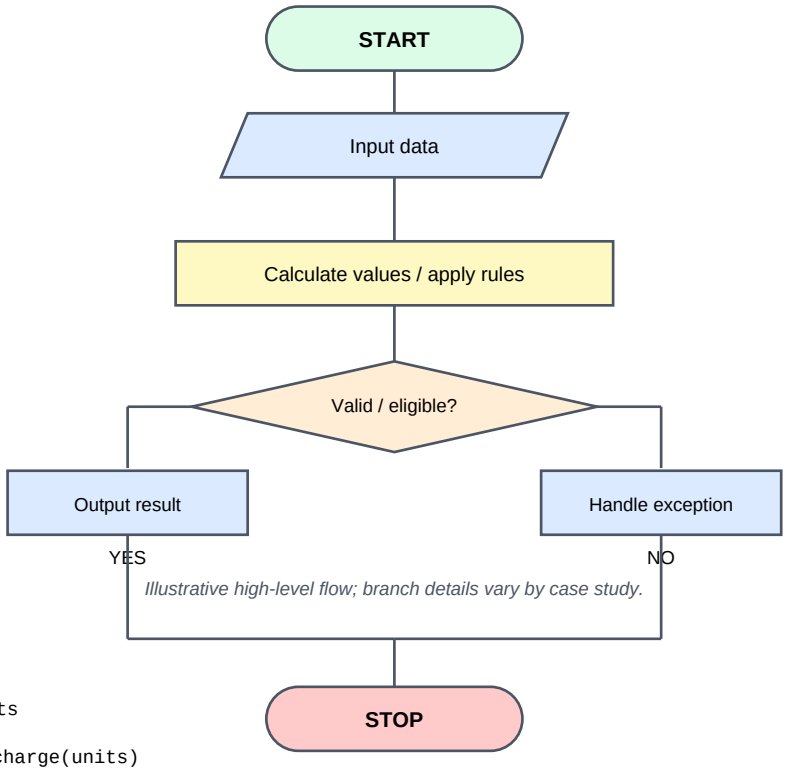
Assumptions

• Units are non-negative integers.
• Rates: first 100 units ■2/unit, next 100 ■3/unit, above 200 ■5/unit.
• Fixed charge ■75; surcharge 5%; rebate ■50 if units < 100.
• An increase over 30% compared with the previous bill is unusual.

Algorithm

1. Read monthly unit values in chronological order.
2. Calculate energy charge by slab.
3. Add fixed charge and surcharge; subtract eligible rebate.
4. Compare each bill with the previous month's bill.
5. Print bills and unusual-increase months.

Flowchart



Illustrative high-level flow; branch details vary by case study.

Pseudocode

```
START
INPUT monthly_units
FOR each month
  energy ← slab_charge(units)
  bill ← (energy + 75) × 1.05
  IF units < 100 THEN bill ← bill - 50
  IF previous bill exists AND bill > previous bill × 1.30 THEN flag month
OUTPUT monthly bills and flags
STOP
```

Python source code (compact)

```
units = [80, 120, 260]
energy = [min(u,100)*2 + max(min(u-100,100),0)*3 + max(u-200,0)*5 for u in units]
bills = [round((e+75)*1.05 - (50 if u < 100 else 0), 2) for e,u in zip(energy, units)]
print("Bills:", bills, "Unusual months:", [i+1 for i in range(1,len(bills)) if bills[i] > bills[i-1]*1.30])
```

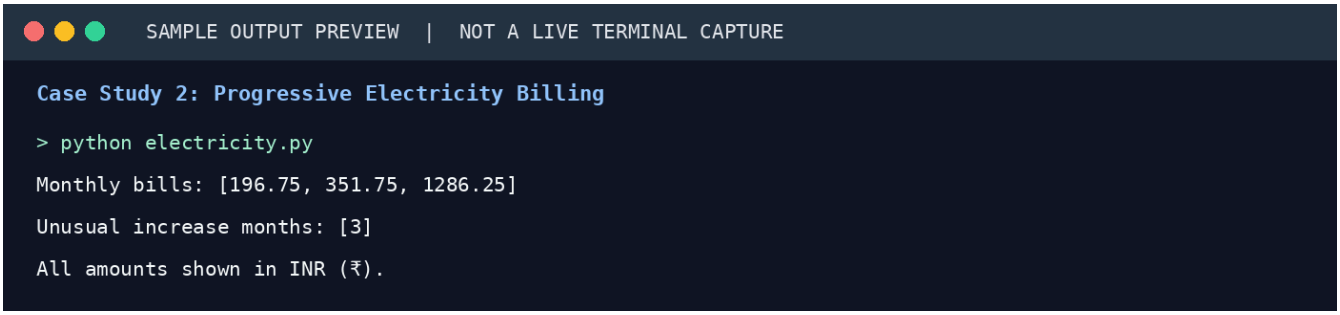
Test cases

Input monthly units	Expected output
80	■196.75

Input monthly units	Expected output
120	■351.75
260	■1,286.25; unusual increase from month 2 to 3

Sample output

```
Bills: [196.75, 351.75, 1286.25] Unusual months: [3]
```



Conclusion

Slab-wise billing gives a transparent monthly total, while comparison logic highlights bills that rise substantially.

3. Hospital Billing and Patient Classification

Problem statement: Prepare a patient bill from consultation, diagnostic and treatment charges, apply concessions, validate amounts and classify the patient's payment status.

Objectives

• Calculate the total bill
• Apply concession rules
• Validate payments and classify account status

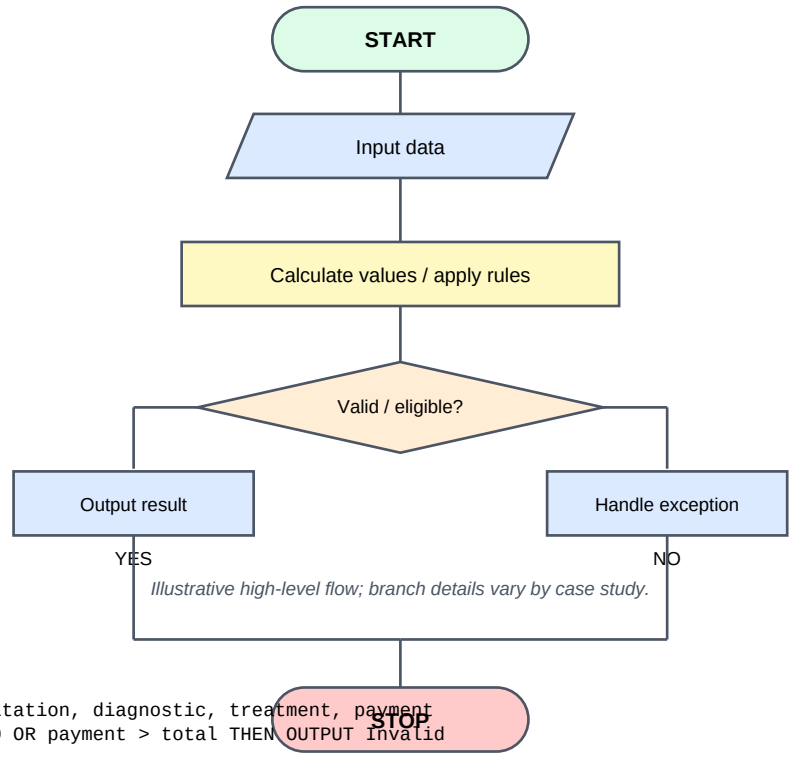
Assumptions

• All charges and payments are non-negative.
• Senior patients (age ≥ 60) receive a 10% concession on consultation only.
• A payment above the bill is invalid; otherwise classify as Paid, Part-paid, or Unpaid.

Algorithm

1. Read age, consultation, diagnostic, treatment and payment.
2. Reject negative amounts or payment above bill.
3. Apply the consultation concession if age is at least 60.
4. Calculate final bill and balance.
5. Classify payment status and display statement.

Flowchart



Illustrative high-level flow; branch details vary by case study.

Pseudocode

```
START
INPUT age, consultation, diagnostic, treatment, payment
IF any amount < 0 OR payment > total THEN OUTPUT Invalid
ELSE
  IF age ≥ 60 THEN consultation ← consultation × 0.90
  total ← consultation + diagnostic + treatment
  balance ← total - payment
  classify status using balance and payment
  OUTPUT total, balance, status
STOP
```

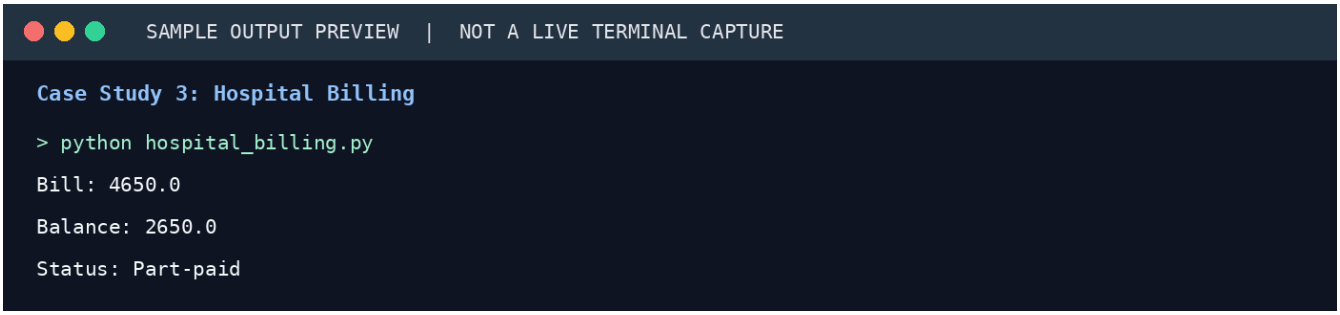
Python source code (compact)

```
age, consult, tests, treatment, paid = 65, 500, 1200, 3000, 2000
if min(consult, tests, treatment, paid) < 0 or paid > consult + tests + treatment: print("Invalid billing data")
else:
 total = consult * (0.9 if age >= 60 else 1) + tests + treatment; balance = total - paid
 print("Bill:", total, "Balance:", balance, "Status:", "Paid" if balance == 0 else "Part-paid" if paid else "Unpaid")
```

Test cases

Inputs	Expected output
Age 65; charges 500,1200,3000; paid 2000	Bill ■4,650; balance ■2,650; Part-paid
Age 40; charges 500,1200,3000; paid 0	Bill ■4,700; balance ■4,700; Unpaid
Payment exceeds bill	Invalid billing data

Sample output



Conclusion

Conditional concessions and validation produce a consistent patient bill and clearly communicate the outstanding balance.

4. Employee Payroll and Deduction Analysis

Problem statement: Calculate gross salary from basic pay, allowances, overtime and incentives; apply progressive deductions and print a salary statement.

Objectives

• Compute gross earnings
• Apply progressive deductions
• Calculate net salary

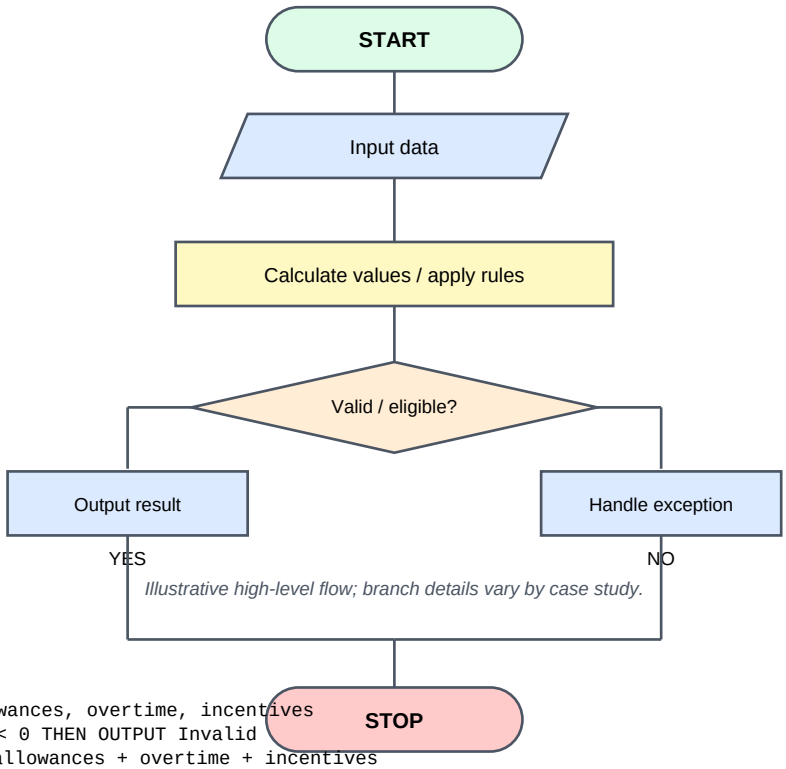
Assumptions

• Basic pay, allowances, overtime and incentives are non-negative.
• Gross salary = basic + allowances + overtime + incentives.
• Deduction is 5% on the first ■20,000 and 10% on the remainder.

Algorithm

1. Read salary components.
2. Validate that all components are non-negative.
3. Calculate gross salary.
4. Calculate progressive deduction.
5. Calculate net salary and display the statement.

Flowchart



Illustrative high-level flow; branch details vary by case study.

Pseudocode

```
START
INPUT basic, allowances, overtime, incentives
IF any component < 0 THEN OUTPUT Invalid
gross ← basic + allowances + overtime + incentives
deduction ← min(gross,20000)×0.05 + max(gross-20000,0)×0.10
net ← gross - deduction
OUTPUT gross, deduction, net
STOP
```

Python source code (compact)

```
basic, allowances, overtime, incentive = 25000, 5000, 2000, 1000
gross = basic + allowances + overtime + incentive
deduction = min(gross, 20000)*0.05 + max(gross-20000, 0)*0.10
print(f"Gross: ■{gross:.2f}\nDeduction: ■{deduction:.2f}\nNet salary: ■{gross-deduction:.2f}")
```

Test cases

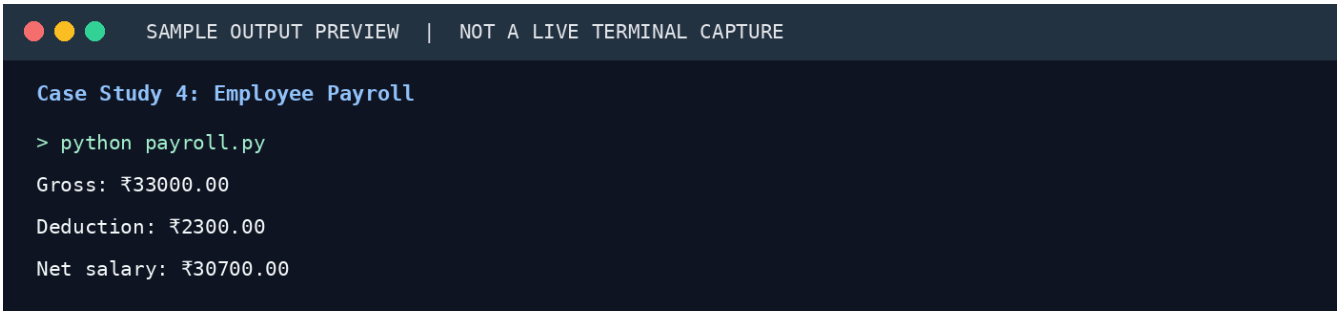
Inputs	Expected output
Basic 25000, allowances 5000, overtime 2000, incentive 1000	Gross ■33000; deduction ■2300; net ■30700

Inputs	Expected output
All components zero	Gross ■0; deduction ■0; net ■0

Sample output

```
Gross: ■33000.00 | Deduction: ■2300.00 | Net salary: ■30700.00
```

Conclusion



Conclusion

The payroll calculation combines earnings and progressive deductions to produce a simple, itemized salary summary.

5. Multi-Plan Telecommunications Cost Optimization

Problem statement: Compare mobile plans against usage, validity and budget, including data/call excess charges, and select the lowest-cost eligible plan.

Objectives

• Calculate plan cost including excess usage
• Check validity and budget
• Identify the cheapest eligible plan

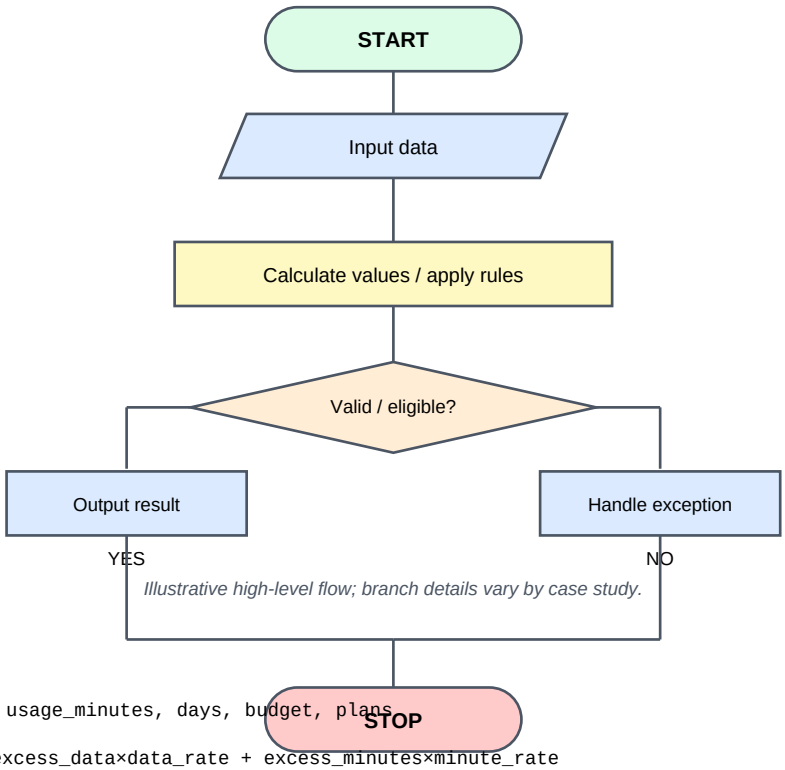
Assumptions

• Each plan tuple is (name, included_data_GB, included_minutes, base_price, validity_days, excess_GB_rate, excess_minute_rate).
• Usage and prices are non-negative.
• Only plans meeting validity and budget are considered.

Algorithm

1. Read usage, required validity and budget.
2. For each plan, calculate excess data and minutes.
3. Calculate total cost.
4. Keep plans within budget and with sufficient validity.
5. Choose the eligible plan with the lowest total cost.

Flowchart



Illustrative high-level flow; branch details vary by case study.

Pseudocode

```
START
INPUT usage_data, usage_minutes, days, budget, plans
FOR each plan
  cost ← base + excess_data×data_rate + excess_minutes×minute_rate
  IF validity ≥ days AND cost ≤ budget THEN add to eligible
IF eligible is empty THEN OUTPUT No plan
ELSE OUTPUT plan with minimum cost
STOP
```

Python source code (compact)

```
plans = [("Basic", 2, 100, 199, 28, 50, 1), ("Plus", 5, 300, 299, 56, 40, 1), ("Max", 10, 500, 499, 84, 20, 0.5)]
data, minutes, days, budget = 4, 250, 28, 400
costs = [(p[0], p[3]+max(data-p[1],0)*p[5]+max(minutes-p[2],0)*p[6]) for p in plans if p[4] >= days]
print(min((x for x in costs if x[1] <= budget), key=lambda x:x[1], default="No eligible plan"))
```

Test cases

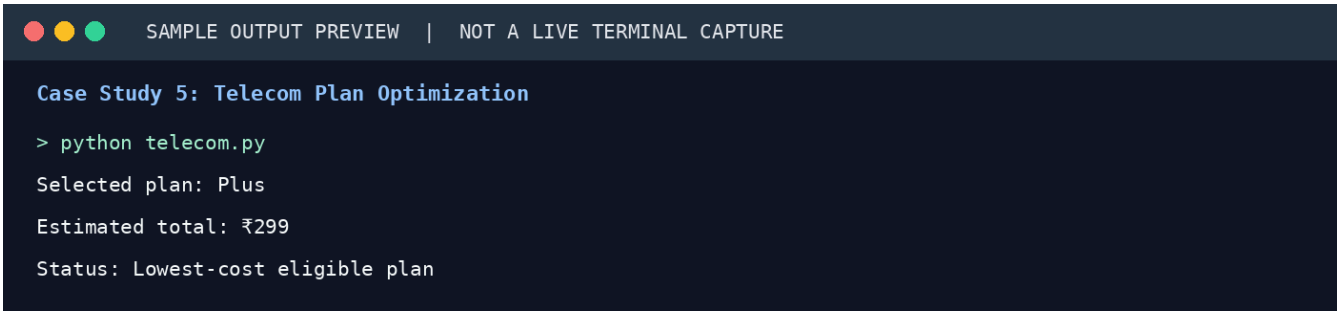
Usage, validity, budget	Expected output
4 GB, 250 minutes, 28 days, ■400	Plus plan, ■299
4 GB, 250 minutes, 28 days, ■250	Basic plan, ■349 exceeds budget; no eligible plan

Sample output

```
('Plus', 299)
```

Conclusion

Plan comparison estimates the total cost for a given usage pattern and identifies the most economical plan that meets the constraints.



Conclusion

Plan comparison estimates the total cost for a given usage pattern and identifies the most economical plan that meets the constraints.

6. Railway Fare and Reservation Calculator

Problem statement: Calculate fares for multiple passengers using distance slabs, travel class, age-based concessions, reservation fees and cancellation penalties.

Objectives

• Calculate distance-based fares
• Apply class multipliers and age concessions
• Estimate cancellation refunds

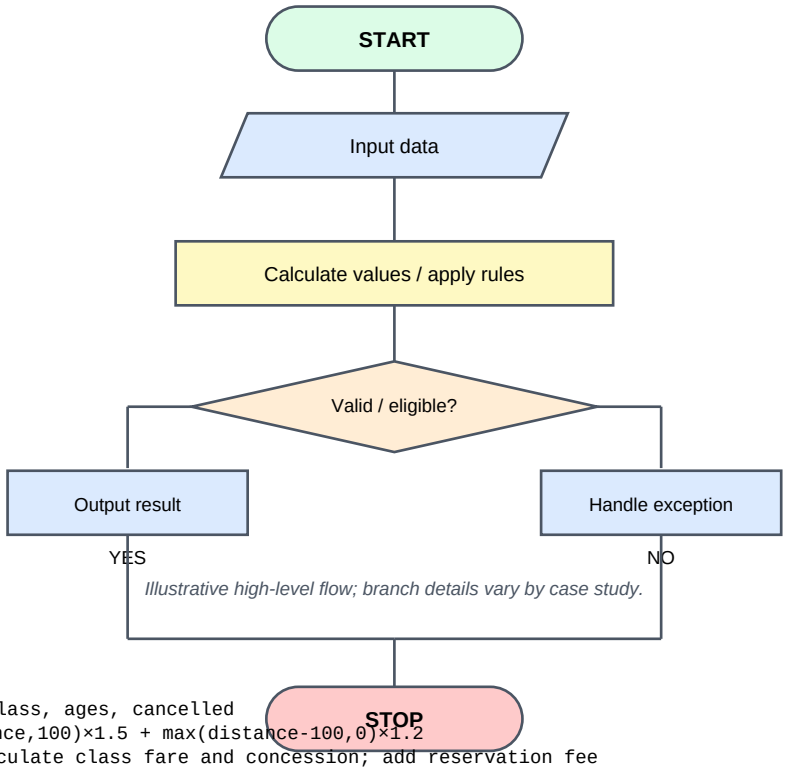
Assumptions

• Base fare: ■1.50 per km for the first 100 km, ■1.20 per km for additional distance.
• Class multipliers: Sleeper 1.0, AC 2.0.
• Children under 5 travel free; passengers aged 60+ get 20% off base fare.
• Reservation fee ■40 per passenger; cancellation penalty 25% of total fare.

Algorithm

1. Read distance, class, passenger ages and cancellation choice.
2. Calculate base fare by distance slab.
3. Apply class multiplier and age concession.
4. Add reservation fee per chargeable passenger.
5. If cancelled, calculate penalty and refund; display totals.

Flowchart



Illustrative high-level flow; branch details vary by case study.

Pseudocode

```
START
INPUT distance, class, ages, cancelled
base ← min(distance,100)×1.5 + max(distance-100,0)×1.2
FOR each age: calculate class fare and concession; add reservation fee
total ← sum(passenger fares)
IF cancelled THEN penalty ← total×0.25; refund ← total-penalty
OUTPUT total, penalty, refund
STOP
```

Python source code (compact)

```
distance, multiplier, ages, cancelled = 180, 2.0, [30, 65, 4], False
base = min(distance,100)*1.5 + max(distance-100,0)*1.2
fares = [0 if a < 5 else base*multiplier*(0.8 if a >= 60 else 1)+40 for a in ages]
total = sum(fares); penalty = total*0.25 if cancelled else 0
print("Fares:", fares, "Total:", total, "Penalty:", penalty, "Refund:", total-penalty if cancelled else 0)
```

Test cases

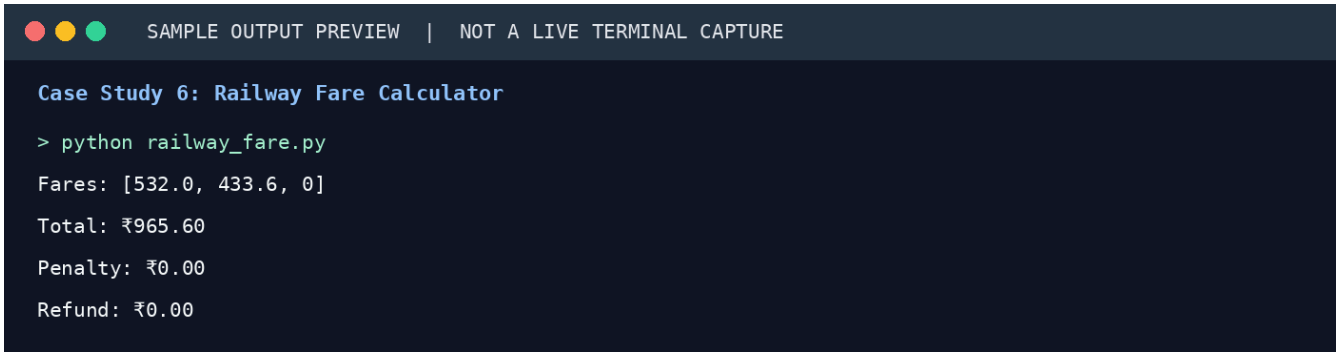
Inputs	Expected output
180 km, AC, ages 30/65/4, not cancelled	Fares ■532, ■433.60, ■0; total ■965.60

Inputs	Expected output
Same trip, cancelled	Penalty ■241.40; refund ■724.20

Sample output

```
Fares: [532.0, 433.6, 0] Total: 965.6 Penalty: 0 Refund: 0
```

Conclusion



Conclusion

Distance slabs, class multipliers and age rules can be combined to estimate passenger fares and cancellation refunds.

7. Retail Billing with Multiple Discount Rules

Problem statement: Generate an itemized retail invoice with product quantities, category taxes, threshold discounts and a maximum concession limit.

Objectives

• Calculate line totals and category taxes
• Apply threshold-based discounts
• Limit discount and compute invoice total

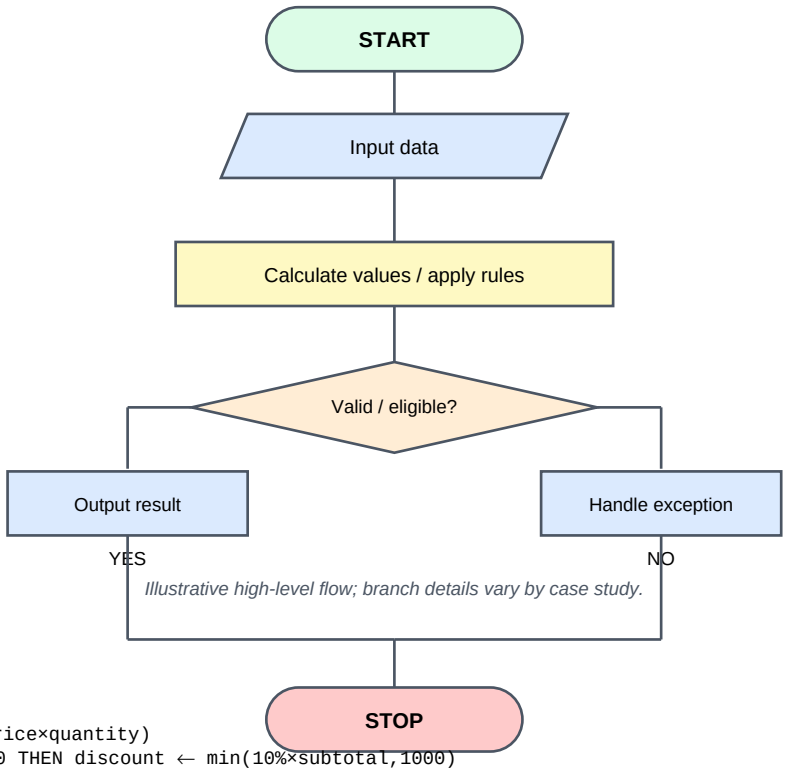
Assumptions

• Product tuples contain (name, price, quantity, tax_rate).
• Subtotal discount: 10% if subtotal ≥ ■5,000; otherwise 5% if subtotal ≥ ■2,000.
• Maximum discount is ■1,000; tax is calculated after discount for this simplified example.

Algorithm

1. Read product tuples.
2. Calculate each line amount and subtotal.
3. Calculate discount based on subtotal and apply the cap.
4. Apply discount proportionally to subtotal, then calculate tax on discounted amount.
5. Display itemized invoice and grand total.

Flowchart



Illustrative high-level flow; branch details vary by case study.

Pseudocode

```
START
INPUT products
subtotal ← sum(price×quantity)
IF subtotal ≥ 5000 THEN discount ← min(10%×subtotal,1000)
ELSE IF subtotal ≥ 2000 THEN discount ← 5%×subtotal ELSE discount ← 0
tax ← (subtotal-discount)×weighted_tax_rate
OUTPUT invoice and grand total
STOP
```

Python source code (compact)

```
items = [("Book", 500, 2, 0.05), ("Bag", 1800, 2, 0.12)]
subtotal = sum(price*qty for name,price,qty,tax in items)
discount = min(subtotal*0.10, 1000) if subtotal >= 5000 else subtotal*0.05 if subtotal >= 2000 else 0
tax_total = sum((price*qty)*(1-discount/subtotal)*tax for name,price,qty,tax in items) if subtotal else 0; print("Subtotal:", s
```

Test cases

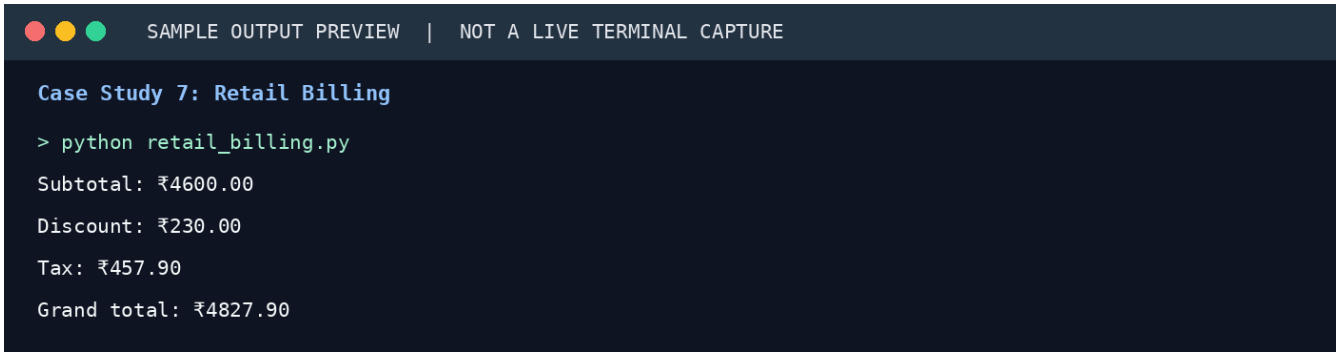
Items	Expected output
2 books at ■500 with 5% tax; 2 bags at ■1800 with 12% tax	Subtotal ■4600; discount ■230; tax ■457.90; total ■4827.90

Items	Expected output
No items	Subtotal ■0; discount ■0; tax ■0; total ■0

Sample output

```
Subtotal: 4600 Discount: 230.0 Tax: 457.9 Total: 4827.9
```

Conclusion



Conclusion

The invoice combines quantities, tax rates and capped discounts to calculate a transparent final amount.

8. Manufacturing Cost and Profitability Analysis

Problem statement: Calculate material, labour, wastage and machine costs, determine unit cost and selling price, and test whether production meets a profit target.

Objectives

• Calculate total production cost
• Include material wastage and operating costs
• Compare actual profit with a target

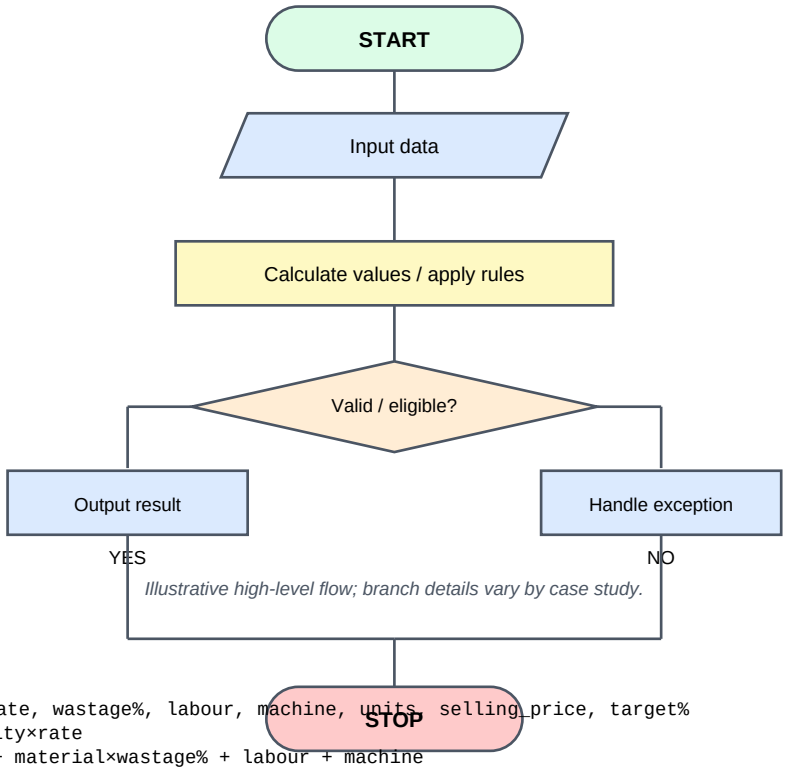
Assumptions

• Material cost is quantity × rate.
• Wastage is a percentage of material cost.
• Total cost includes labour and machine cost.
• Profit target is supplied as a percentage of total cost.

Algorithm

1. Read material quantity, rate, wastage percentage, labour, machine cost, units produced, selling price and target profit percentage.
2. Calculate material cost plus wastage.
3. Calculate total cost and unit cost.
4. Calculate revenue and profit percentage.
5. Compare profit percentage with target and display result.

Flowchart



Illustrative high-level flow; branch details vary by case study.

Pseudocode

```
START
INPUT quantity, rate, wastage%, labour, machine, units, selling_price, target%
material ← quantity×rate
cost ← material + material×wastage% + labour + machine
revenue ← units×selling_price; profit ← revenue-cost
profit% ← profit/cost×100
OUTPUT cost, unit cost, profit, target result
STOP
```

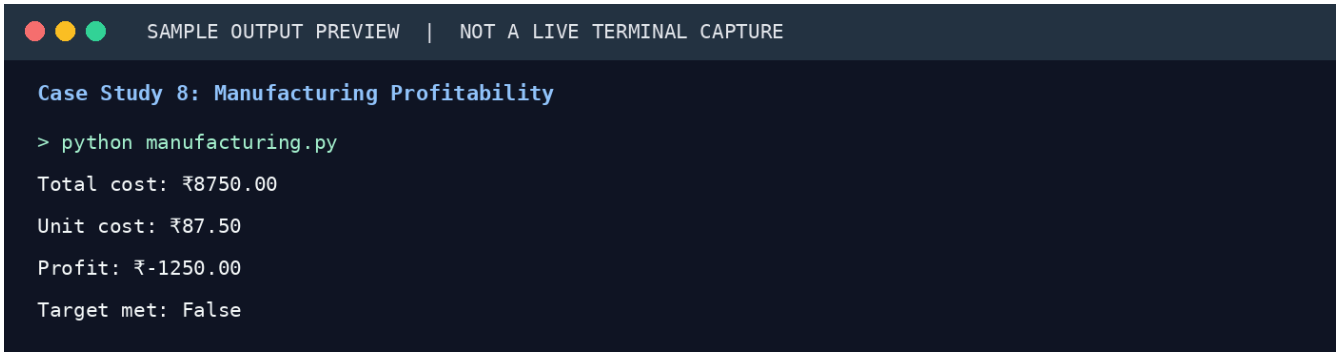
Python source code (compact)

```
qty, rate, waste, labour, machine, units, price, target = 100, 50, 5, 2000, 1500, 100, 75, 20
material = qty*rate; total_cost = material + material*waste/100 + labour + machine
revenue = units*price; profit = revenue-total_cost
print("Cost:", total_cost, "Unit cost:", round(total_cost/units,2), "Profit:", profit, "Target met:", profit >= total_cost*targ
```

Test cases

Inputs	Expected output
100 kg × ■50; wastage 5%; labour ■2000; machine ■1500; 100 units at ■75; target 20%	Cost ■8,750; unit cost ■87.50; profit –■1,250; target not met
Same costs, selling price ■120	Revenue ■12,000; profit ■3,250; target met

Sample output



Conclusion

Cost and revenue analysis identifies loss-making production and checks whether the required profit margin has been achieved.

9. Multi-Currency Travel Budget Analysis

Problem statement: Convert travel expenses using supplied exchange rates, include conversion fees, compare total spending with a budget, and identify the largest expense category.

Objectives

• Convert each expense to home currency
• Add conversion fees
• Compare spending with budget and find the largest category

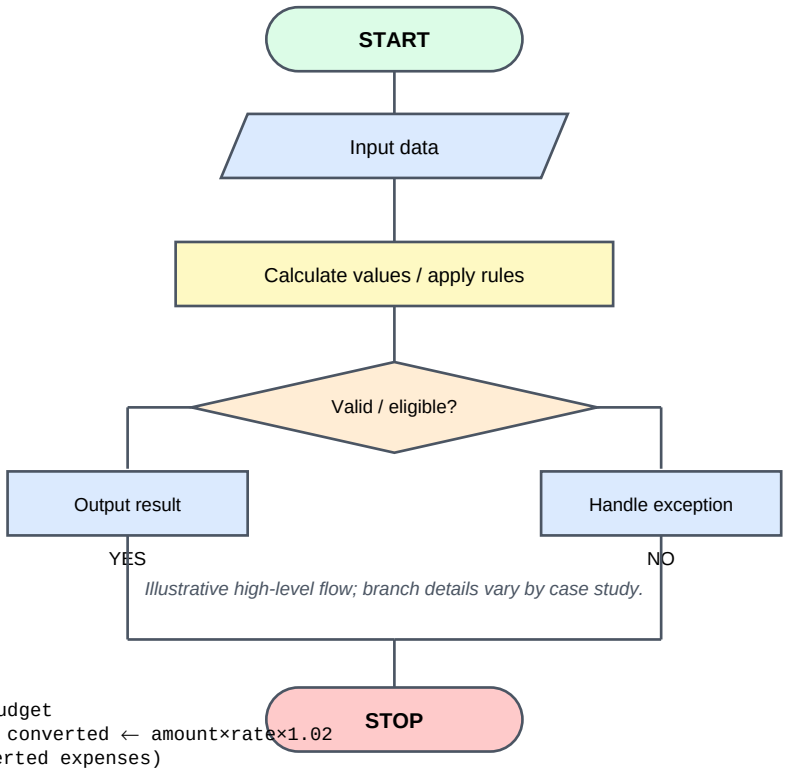
Assumptions

• Each expense tuple is (category, foreign_amount, exchange_rate).
• A 2% conversion fee applies to converted amounts.
• Budget is expressed in home currency.

Algorithm

1. Read expense tuples and budget.
2. Convert each foreign amount using its exchange rate.
3. Add conversion fee to each converted amount.
4. Calculate total and identify largest converted expense.
5. Compare total with budget and display results.

Flowchart



Illustrative high-level flow; branch details vary by case study.

Pseudocode

```
START
INPUT expenses, budget
FOR each expense: converted ← amount×rate×1.02
total ← sum(converted expenses)
largest ← category with maximum converted expense
IF total ≤ budget THEN OUTPUT Within budget ELSE OUTPUT Over budget
OUTPUT total and largest category
STOP
```

Python source code (compact)

```
expenses = [("Hotel", 200, 83), ("Food", 80, 83), ("Transport", 50, 83)]
converted = [(name, round(amount*rate*1.02,2)) for name,amount,rate in expenses]
total, budget = sum(x[1] for x in converted), 30000
print("Expenses:", converted, "Total:", total, "Largest:", max(converted,key=lambda x:x[1])[0], "Status:", "Within budget" if t
```

Test cases

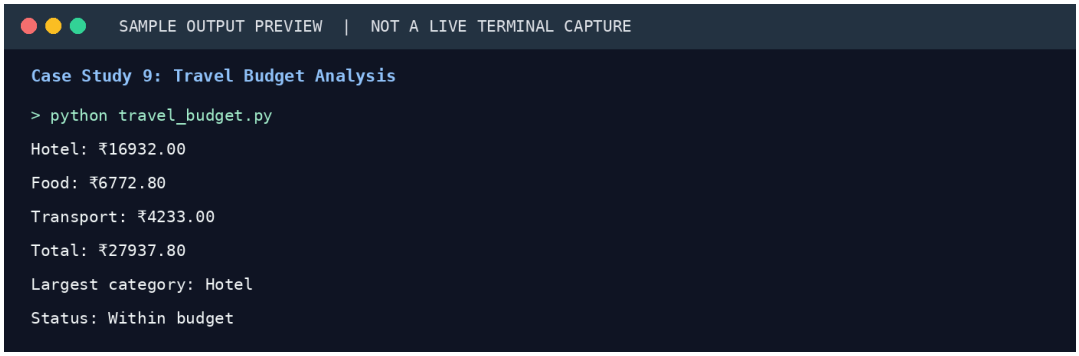
Expenses and budget	Expected output
Hotel 200 units, food 80, transport 50; rate 83; budget ■30000	Total ■27,937.80; largest Hotel; Within budget

Expenses and budget	Expected output
Same expenses; budget ■20000	Over budget

Sample output

```
Expenses: [('Hotel', 16932.0), ('Food', 6772.8), ('Transport', 4233.0)] Total: 27937.8 Largest: Hotel Status: Within budget
```

Conclusion



Conclusion

Currency conversion with a fee provides a consistent home-currency estimate and highlights the largest budget category.

10. Campus Event Financial Planning

Problem statement: Calculate event costs for venue, equipment, catering, transport and certificates; determine participant contributions, sponsorship shortfall and break-even registrations.

Objectives

• Calculate total event expenditure
• Estimate contribution per participant and sponsorship shortfall
• Calculate minimum registrations to break even

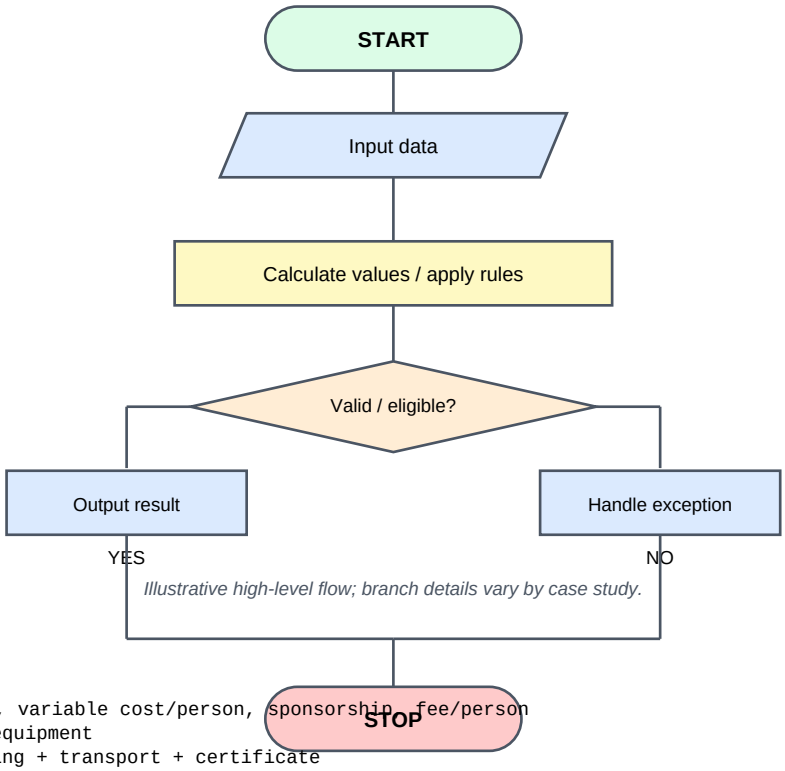
Assumptions

• Venue and equipment are fixed costs.
• Catering, transport and certificates are per-participant costs.
• Sponsorship is a fixed contribution.
• Registration fee is fixed per participant.

Algorithm

1. Read fixed costs, per-person costs, sponsorship and registration fee.
2. Calculate total cost for the expected number of participants.
3. Calculate remaining amount after sponsorship.
4. Calculate contribution per participant if needed.
5. Compute minimum registrations required to cover fixed costs after sponsorship and variable costs.

Flowchart



Illustrative high-level flow; branch details vary by case study.

Pseudocode

```
START
INPUT fixed costs, variable cost/person, sponsorship, fee/person
fixed ← venue + equipment
variable ← catering + transport + certificate
net_fixed ← max(fixed-sponsorship,0)
break_even ← ceiling(net_fixed/(fee-variable)) if fee > variable else impossible
OUTPUT total cost, shortfall, break_even
STOP
```

Python source code (compact)

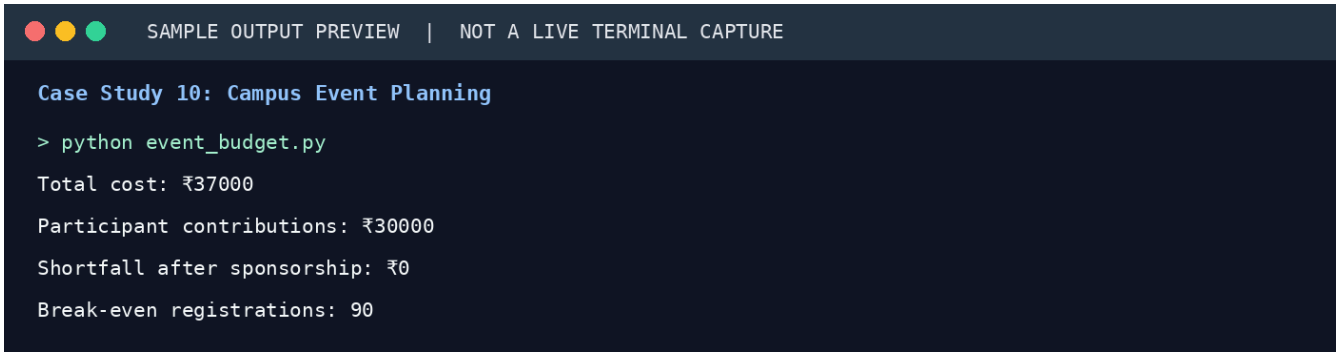
```
import math
venue, equipment, catering, transport, certificate, sponsor, fee, participants = 12000, 5000, 150, 40, 10, 8000, 300, 100
fixed = venue + equipment; variable = catering + transport + certificate; total = fixed + variable*participants
break_even = math.ceil(max(fixed-sponsor,0)/(fee-variable)) if fee > variable else 'Impossible'; print('Total:', total, 'Contri
```

Test cases

Inputs	Expected output
Fixed costs ■17000; variable ■200/person; sponsor ■8000; fee ■300; 100 participants	Total ■37,000; contributions ■30,000; shortfall ■0 after sponsor; break-even 90
Fee is less than variable cost per participant	Break-even impossible

Sample output

```
Total cost: 37000 Particpant contribution: 30000 Shortfall: 0 Break-even registrations: 90
```



Conclusion

The event budget distinguishes fixed and variable costs and estimates the registrations needed to cover costs after sponsorship.